A Mini Project Report on

PLANT DOCTOR

A Multilingual Voice-Enabled Dual-Index RAG Chatbot for Agricultural Advisory and Government Scheme Assistance

Submitted to Anurag University in partial fulfilment of the requirements

for the award of the Degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

Submitted by

Chunku Raja Venkata Praney Teja — 23EG106C10

Prasadam Lakshmi Narasimha — 23EG106C49

Konda Mukesh — 23EG106C31

Under the Guidance of

Mr. G. Victor Daniel

Assistant Professor, Department of Artificial Intelligence



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

SCHOOL OF ENGINEERING

ANURAG UNIVERSITY, HYDERABAD

Academic Year 2026–27 | November 2026

A Mini Project Report on

PLANT DOCTOR

A Multilingual Voice-Enabled Dual-Index RAG Chatbot for Agricultural Advisory and Government Scheme Assistance

Submitted to Anurag University in partial fulfilment of the requirements

for the award of the Degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

Submitted by

Chunku Raja Venkata Praney Teja — 23EG106C10

Prasadam Lakshmi Narasimha — 23EG106C49

Konda Mukesh — 23EG106C31

Under the Guidance of

Mr. G. Victor Daniel

Assistant Professor, Department of Artificial Intelligence



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

SCHOOL OF ENGINEERING

ANURAG UNIVERSITY, HYDERABAD

Academic Year 2026–27 | November 2026

ANURAG UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the Mini Project report entitled "PLANT DOCTOR: A Multilingual Voice-Enabled Dual-Index RAG Chatbot for Agricultural Advisory and Government Scheme Assistance" is a bonafide record of the work carried out by Chunku Raja Venkata Praney Teja (23EG106C10), Prasadam Lakshmi Narasimha (23EG106C49), Konda Mukesh (23EG106C31), in IV B.Tech I Semester, Artificial Intelligence, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence, under our supervision and guidance. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

The project work carried out by the above candidates is approved for submission.

Name of the Student	Roll No.	Signature
Chunku Raja Venkata Praney Teja	23EG106C10	
Prasadam Lakshmi Narasimha	23EG106C49	
Konda Mukesh	23EG106C31	

Project Guide	Head of the Department	External Examiner
Mr. G. Victor Daniel Assistant Professor	Dr. M. Trupti Professor & Head, Dept. of AI	

DECLARATION

We hereby declare that the Mini Project report entitled "PLANT DOCTOR: A Multilingual Voice-Enabled Dual-Index RAG Chatbot for Agricultural Advisory and Government Scheme Assistance", submitted to the Department of Artificial Intelligence, Anurag University, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence, is a record of original work carried out by us under the guidance of Mr. G. Victor Daniel, Assistant Professor, Department of Artificial Intelligence. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma. Wherever the work of others — including documents, pretrained models and publicly available code — has been used, it has been duly acknowledged and cited. The results reported are genuine and reproducible from the submitted code.

Use of AI tools: We declare that we used an AI assistant (Claude) during this project for software-development help and for language editing of this report. All experiments were run by us, and every result reported here was checked against the logged output of the system. We take full responsibility for the content of this report.

Place: Hyderabad

Date: _____

Name of the Student	Roll No.	Signature
Chunku Raja Venkata Praney Teja	23EG106C10	
Prasadam Lakshmi Narasimha	23EG106C49	
Konda Mukesh	23EG106C31	

ACKNOWLEDGEMENT

We are grateful to Prof. Archana Mantri, Vice-Chancellor, Anurag University, for the facilities the University made available to us and for the environment in which this project could be carried out.

We thank Dr. V. Vijay Kumar, Dean, School of Engineering, for his encouragement throughout our course of study.

We sincerely thank Dr. M. Trupti, Professor and Head of the Department of Artificial Intelligence, for providing the resources and support we needed to complete this work.

We are especially indebted to our guide, Mr. G. Victor Daniel, Assistant Professor, Department of Artificial Intelligence. The questions raised during the project reviews shaped how we tested the system, and the feedback on each draft improved this report.

We also thank the developers of the open-source libraries this project is built on — in particular FAISS, sentence-transformers, fastembed, Faster-Whisper, Edge-TTS, FastAPI, LangChain and Ollama — and the publishers of the agricultural and government-scheme documents that form our knowledge base.

Chunku Raja Venkata Praney Teja (23EG106C10)

Prasadam Lakshmi Narasimha (23EG106C49)

Konda Mukesh (23EG106C31)

ABSTRACT

Smallholder farmers need two different kinds of help: practical crop advice — what is wrong with a plant and what to do about it — and administrative guidance on government schemes, subsidies and insurance they qualify for. A single large language model can answer both fluently, but it may draw on the wrong domain or invent details, and language and literacy barriers keep many farmers from using it at all.

This project presents Plant Doctor, a multilingual, voice-enabled, dual-index Retrieval-Augmented Generation (RAG) assistant. Two separate knowledge bases — one for agricultural diagnostics and one for government schemes — each keep their own FAISS dense index, BM25 sparse index and SQLite metadata store, which keeps the two domains from contaminating each other. Retrieval fuses dense cosine similarity with BM25, applies Maximal Marginal Relevance for diversity, and reorders the result with a BAAI/bge-reranker-base Cross-Encoder before a grounded prompt is sent to the language model. The generation layer is provider-agnostic: Ollama (qwen2.5:7b) locally or Groq (gpt-oss-120b) in the cloud. A multilingual pipeline covers English, Hindi, Telugu and Tamil using script-based language detection, adaptive translation, Faster-Whisper speech recognition and Edge-TTS speech synthesis, delivered through a custom web interface.

On a measured benchmark over the agricultural corpus (local qwen2.5:7b, CPU), hybrid retrieval reaches 75.4% context precision with 100% context recall, and the full proposed pipeline (hybrid + reranker) answers with 79.2% faithfulness at a mean end-to-end latency of 5.59 s. The retrieval and generation framework is fully reproducible from the submitted code.

The main value of the architecture is domain isolation plus hybrid retrieval, which improve grounding and precision, and a multilingual voice layer that makes the information reachable in a farmer's own language by text or speech.

Keywords: Retrieval-Augmented Generation, Hybrid Retrieval, BM25, FAISS, Cross-Encoder Reranking, Multilingual NLP, Voice Chatbot, Agricultural AI, Government Schemes.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1

1.1 Background / Overview 1

1.2 Motivation 3

1.3 Problem Statement 3

1.4 Objectives 4

1.5 Scope and Limitations 4

1.6 Organization of the Report 5

CHAPTER 2: LITERATURE SURVEY 6

2.1 Review of Related Work 6

2.1.1 Retrieval-Augmented Generation 6

2.1.2 Sparse Retrieval and Hybrid Fusion 7

2.1.3 Reranking and Diversity 8

2.1.4 Multilingual, Voice and Agricultural Assistants 8

2.1.5 Evaluating Retrieval-Augmented Systems 9

2.2 Summary Table 9

2.3 Research Gap Identified 10

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS 12

3.1 Existing Approaches and their Drawbacks 12

3.2 Feasibility Study 12

3.3 Hardware / Compute Used 13

3.4 Software Requirements 14

3.5 Functional and Non-Functional Requirements 14

CHAPTER 4: DATASET, DESIGN AND METHODOLOGY 16

4.1 Overall Pipeline 16

4.2 Dataset: Knowledge Base 17

4.3 Pre-processing and Chunking 18

4.4 Train / Validation / Test Protocol 18
4.5 System Architecture and Justification 19
4.6 Methodology 20
4.6.1 Dense Retrieval 20
4.6.2 Sparse BM25 Retrieval 20
4.6.3 Hybrid Fusion 20
4.6.4 Maximal Marginal Relevance 20
4.6.5 Cross-Encoder Reranking 21
4.6.6 Grounded Generation and Confidence 21
4.6.7 Multilingual Routing and Voice 21
4.6.8 Worked Retrieval Example 22
4.6.9 Ingestion and Indexing 23
4.6.10 Prompt Design and Grounding Constraints 23
4.6.11 Caching and Performance 23
4.7 Objective-to-Methodology Mapping 24
4.8 Baseline and Comparison Methods 25
CHAPTER 5: IMPLEMENTATION 26
5.1 System / Pipeline Configuration 26
5.2 Development and Deployment Setup 27
5.3 Key Code Snippets 28
5.4 User Interface and Deployment 28
5.5 Cloud Deployment and Provider Switching 29
5.6 REST API 30
5.7 Testing and Validation 30
CHAPTER 6: RESULTS AND DISCUSSION 32
6.1 Evaluation Metrics 32
6.2 Retrieval Ablation (Measured) 33

6.3 Generation and Latency (Measured) 33
6.4 Comparison with Baselines and Published Work 35
6.5 Error Analysis 36
6.6 Ablation Summary 36
6.7 Output of the Working System 37
6.8 Ethical and Responsible AI Considerations 38
CHAPTER 7: CONCLUSION AND FUTURE SCOPE 40
7.1 Conclusion 40
7.2 Limitations 40
7.3 Future Enhancements 41
REFERENCES 42
APPENDICES 44
Appendix A GitHub Repository and README 44
Appendix B Models and Indices 44
Appendix C Key Commands to Reproduce 44
Appendix D Plagiarism Report 44
Appendix E Publications / Associated Work 45

LIST OF FIGURES

Fig. No.	Title
4.1	Layered System Architecture of Plant Doctor
4.2	End-to-End Query Processing Pipeline
4.3	Dual-Index Domain Isolation
6.1	Retrieval Ablation: Context Precision and Recall
6.2	Per-Stage Latency Breakdown
6.3	Generation Quality: Direct LLM vs Proposed RAG
6.4	Chat Assistant — Grounded Answer with Citations (Screenshot)
6.5	Government Schemes Page — Second Knowledge Domain (Screenshot)
6.6	Document Library — Indexed Documents (Screenshot)

Note: page numbers for figures/tables finalise automatically when the document fields are updated in Word (select all, press F9).

LIST OF TABLES

Table No.	Title
2.1	Summary of Related Work and Research Gap
3.1	Software Requirements and Development Environment
4.1	Knowledge-Base (Corpus) Summary
4.2	Objective-to-Methodology Mapping
5.1	System / Pipeline Configuration
5.2	REST API Endpoints Exposed by the Backend
6.1	Retrieval Ablation Results (Measured)
6.2	Generation and Latency Results (Measured)

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BM25	Best Match 25 (Okapi ranking)
CPU	Central Processing Unit
FAISS	Facebook AI Similarity Search
GPU	Graphics Processing Unit
IDF	Inverse Document Frequency
LLM	Large Language Model
MMR	Maximal Marginal Relevance
NLI	Natural Language Inference
ONNX	Open Neural Network Exchange
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SPA	Single-Page Application
TF	Term Frequency
TTS	Text-to-Speech
TYLCV	Tomato Yellow Leaf Curl Virus
UI	User Interface
WER	Word Error Rate

CHAPTER 1: INTRODUCTION

1.1 Background / Overview

Agriculture is an information-intensive occupation and the backbone of the Indian economy. Whether a farmer catches a disease outbreak early, applies the right fertiliser dose, or claims a subsidy before its deadline often comes down to whether the right information arrived in time. The same farmer may need agronomic advice in the morning and scheme paperwork guidance in the afternoon, and the two kinds of question carry very different semantics and risks. A crop-diagnosis question deals in symptoms, treatments, pesticides and fertiliser recommendations; a scheme question deals in eligibility rules, required documents, benefit amounts, deadlines and application steps [1][2].

Large Language Models (LLMs) such as GPT, Gemini, Llama and Qwen have made it easy to ask such questions in plain language, and their answers are fluent and confident. The difficulty is that fluency is not accuracy: these models can hallucinate, producing statements that sound authoritative but are not supported by any source [3]. In an agricultural setting a confidently wrong answer about a pesticide dose or a scheme's eligibility can cause real financial harm. Retrieval-Augmented Generation (RAG) [4] addresses this by giving the model passages retrieved from a trusted document collection and asking it to answer from them, which improves grounding and lets every answer be traced back to its source.

This project builds Plant Doctor, a conversational assistant that combines RAG with multilingual and voice interaction, organised around two deliberately isolated domains: agricultural diagnostics and government schemes. Keeping the domains apart reduces retrieval contamination between two technically unrelated knowledge sources, while the farmer still talks to a single, simple interface in English, Hindi, Telugu or Tamil, by text or speech.

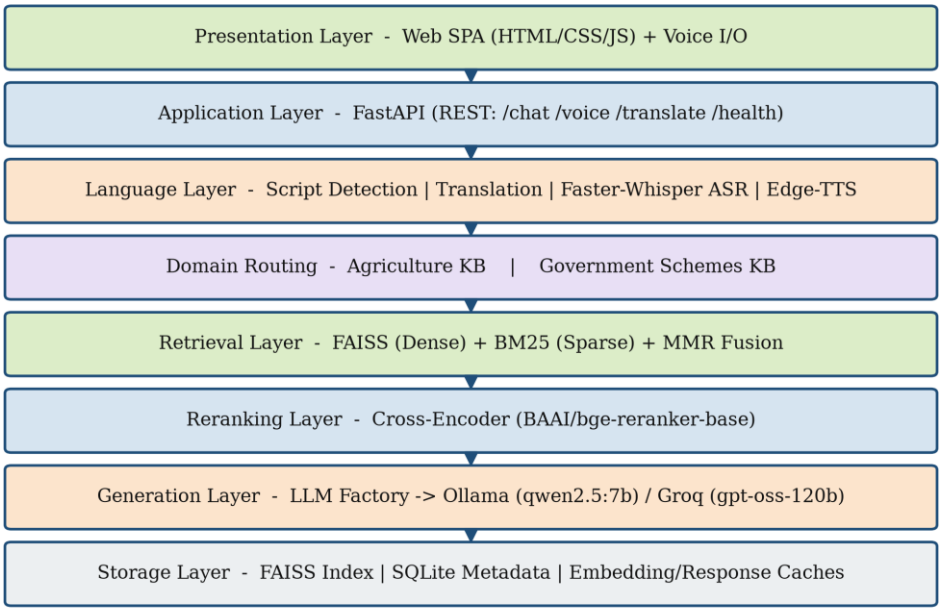


Fig. 1.1 High-level layered architecture of the Plant Doctor system.

India has more than one hundred million operational landholdings, the large majority of them small and marginal. For these farmers, timely and correct information is one of the cheapest inputs available and one of the hardest to obtain. Public agricultural research bodies such as the Indian Council of Agricultural Research (ICAR) and the network of Krishi Vigyan Kendras (KVKs) publish a large body of trustworthy material — crop calendars, pest and disease management guides, fertiliser recommendations and advisories — and the central and state governments publish detailed guidelines for dozens of welfare schemes. In principle every answer a farmer needs is already written down. In practice this knowledge sits in hundreds of separate PDFs and web portals, is written almost entirely in English, and is organised for officials and researchers rather than for a farmer holding a phone in a field. The gap this project addresses is therefore not a lack of knowledge but a lack of access to it in the right form, the right language and at the right time.

Conversational AI is an attractive way to close that gap because it lets a user ask a question in natural language instead of searching documents. The challenge is that a bare language model does not know which document a fact came from, cannot be audited, and will sometimes state something that no source supports. Retrieval-Augmented Generation changes the contract: the model must answer from passages

that were actually retrieved from a trusted collection, and those passages can be shown to the user alongside the answer. This project takes that idea and adds the two things a real farmer-facing deployment needs — separation of unrelated knowledge domains so that answers are not built from the wrong source, and a multilingual voice layer so that the service is usable by someone who prefers to speak Telugu, Hindi or Tamil rather than type English.

1.2 Motivation

Three observations motivated this work. First, the consequences of a wrong agricultural answer are asymmetric: a correct answer saves a farmer some time, but a wrong answer that the farmer trusts can lead to crop loss, wasted inputs or a missed subsidy deadline. A system that is sometimes silent but rarely confidently wrong is therefore more useful in this domain than one that always answers.

Second, trustworthy agricultural knowledge already exists in ICAR and KVK manuals, research journals and official government-scheme guidelines, but it is scattered across PDFs and portals and written mostly in English. Farmers cannot easily search it, and generic chatbots do not ground their answers in it. A document-grounded assistant that cites its sources closes this gap.

Third, accessibility is a problem distinct from answer quality. Many farmers prefer a regional language and prefer speaking to typing. An assistant that accepts voice input, detects the language automatically, and replies in the same language by text and speech removes a barrier that text-only, English-only systems leave in place.

1.3 Problem Statement

Existing agricultural question-answering systems built on single-index RAG retrieve semantically similar passages from across their whole corpus, so a crop-disease query can surface government-scheme passages and vice-versa; they often miss exact policy terms, scheme names and crop-specific keywords that lexical search handles well; they are usually text-only and English-only; and they tend to answer even when the evidence is weak. In particular, this project addresses the following problems:

1. Domain contamination: a single index mixes two unrelated knowledge sources, so answers can be built from the wrong domain.

2. Weak lexical recall: pure dense retrieval misses exact disease names, fertiliser ratios, scheme names and document identifiers that farmers actually use.
3. Accessibility barrier: most systems are text-only and English-only, excluding farmers who prefer regional languages or voice.
4. Ungrounded answers: without retrieval and citation, answers may be fluent but unsupported, which is unsafe for agricultural and financial decisions.

Hence there is a need for a farmer-facing assistant that (1) isolates the agricultural and government-scheme domains into separate indices, (2) retrieves with hybrid dense-plus-sparse search and reranking, (3) understands and replies in the farmer's language by text or voice, and (4) grounds every answer in retrieved documents and shows the sources.

1.4 Objectives

The objectives of this project, stated as in the associated research paper, are:

5. O1 — Design a domain-isolated dual-index RAG architecture for agricultural advisory and government-benefit navigation.
6. O2 — Combine dense semantic retrieval with sparse BM25 retrieval to improve candidate recall and precision.
7. O3 — Apply Cross-Encoder reranking to improve the ordering of retrieved evidence.
8. O4 — Support English, Hindi, Telugu and Tamil through adaptive multilingual query processing.
9. O5 — Provide voice input (speech-to-text) and voice output (text-to-speech).
10. O6 — Establish an evaluation framework covering faithfulness, context precision, context recall, answer relevance and system latency.

Each objective is mapped to a concrete methodology in Chapter 4 (Table 4.2) and its outcome is reported against measured results in Chapter 6.

1.5 Scope and Limitations

Scope. The system covers four languages (English, Hindi, Telugu, Tamil); two knowledge domains (agricultural diagnostics and government schemes); document formats PDF, DOCX, TXT and Markdown; text and voice interaction; and both local

(Ollama) and cloud (Groq) inference. It provides grounded answers with citations, a confidence estimate, latency diagnostics and spoken output through a web interface.

Limitations. Answer quality is bounded by the completeness and freshness of the underlying documents. On very small local models (qwen2.5:7b) direct generation in Indian languages can degrade, so the system falls back to generate-in-English-then-translate on such models. The free translation services are rate-limited, for which an LLM translation fallback is provided. Speech recognition quality depends on audio conditions, and government-scheme information changes over time and needs periodic re-ingestion. The measured benchmark in this report is small (four questions) and run on a local CPU model; it establishes reproducibility rather than large-scale accuracy.

The system is positioned as a decision-support tool for farmers and agricultural extension workers, not as a replacement for a qualified agronomist or an official scheme authority. It answers from the documents it has been given and shows its sources, so the user remains in control of the final decision. Pesticide, fertiliser and financial actions in particular should be confirmed against current local recommendations, and this is made explicit to the user. Within these bounds, the contribution is a working, reproducible system that demonstrates how domain isolation, hybrid retrieval, reranking and a multilingual voice layer can be combined into one farmer-facing assistant.

1.6 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 surveys related work in RAG, hybrid retrieval, reranking and multilingual agricultural chatbots, and identifies the research gap. Chapter 3 analyses existing approaches and states the hardware and software requirements. Chapter 4 describes the corpus, the overall design, the mathematical methodology and the mapping of each objective to its method. Chapter 5 covers the implementation, configuration, key code and deployment. Chapter 6 reports the measured results, ablations, error analysis and responsible-AI considerations. Chapter 7 concludes and lists future enhancements.

CHAPTER 2: LITERATURE SURVEY

2.1 Review of Related Work

This section reviews work in four strands that the project builds on: retrieval-augmented generation, dense and sparse retrieval and their fusion, Cross-Encoder reranking, and multilingual/voice agricultural assistants.

2.1.1 Retrieval-Augmented Generation

Lewis et al. [4] introduced Retrieval-Augmented Generation, coupling a retriever to a generator so that answers are conditioned on retrieved passages rather than on model memory alone. This reduces hallucination and lets answers be traced to sources, but it does not by itself guarantee that the retrieved passages come from the correct domain, which motivates the dual-index design used here.

Karpukhin et al. [5] proposed Dense Passage Retrieval, showing that dual-encoder dense retrieval outperforms traditional sparse methods on open-domain question answering. Dense retrieval matches meaning rather than exact words, but it can miss specialised terminology — exactly the crop names, fertiliser ratios and scheme identifiers that matter in this application.

Since the original formulation, RAG has been extended in many directions — iterative and multi-hop retrieval, query rewriting, adaptive retrieval that decides when to retrieve, and post-hoc verification of the generated answer. These variants trade additional latency and complexity for higher precision on hard questions. For a farmer-facing assistant on constrained infrastructure, the more valuable levers are the ones that improve the first retrieval and keep the answer grounded without extra model calls, which is why this project invests in hybrid retrieval, diversity and reranking rather than in multi-round retrieval loops. The design is nonetheless compatible with those extensions, which are noted as future work.

A recurring weakness of the basic RAG recipe is that retrieval quality caps answer quality: if the right passage is not retrieved, no amount of prompting will make the answer correct, and if an irrelevant passage is retrieved the model may be led astray. Two levers therefore matter most in practice — the retrieval method (how candidates are found) and the ranking method (how the best candidates are placed at the top of

the context window, where the model attends to them most). The survey of hallucination in natural language generation by Ji et al. [3] makes the same point from the opposite direction: grounding reduces but does not remove hallucination, so the quality and ordering of the grounding evidence is decisive. This project treats retrieval and ranking as first-class components rather than as a single black box, which is why it combines sparse, dense, diversity and reranking stages explicitly.

2.1.2 Sparse Retrieval and Hybrid Fusion

Robertson and Zaragoza [6] formalised the Okapi BM25 ranking function, the standard term-frequency/inverse-document-frequency sparse model that still excels at exact keyword matching. Because dense and sparse methods fail in complementary ways, hybrid retrieval that fuses the two is now common practice; Cormack et al. [7] showed that simple rank fusion reliably improves over either signal alone. This project fuses normalised BM25 with dense cosine similarity.

Xiao et al. [8] released the BGE family of embeddings (BAAI/bge-small-en-v1.5), compact 384-dimensional sentence embeddings that give strong retrieval quality at low memory cost, which is why they are used here for both local and cloud deployment. Johnson et al. [9] described FAISS, the similarity-search library used for the dense index; FAISS provides exact and approximate nearest-neighbour search that scales from thousands to billions of vectors, and its flat index is more than sufficient for a corpus of a few thousand chunks while searching in a few milliseconds.

An important practical insight from the hybrid-retrieval literature is that the two signals must be made comparable before they are combined. Cosine similarity is bounded in a small range, whereas BM25 scores are unbounded and depend on corpus statistics, so naïvely adding them lets BM25 dominate. The common remedy, adopted here, is to normalise the BM25 score against the maximum score among the current candidates before a weighted sum. This keeps the fusion weight meaningful and lets a single parameter α trade semantic matching against exact-term matching. In an agricultural setting this trade-off is not academic: a query like 'urea dose for paddy' needs the exact token 'urea' (sparse strength) as well as the concept of nitrogen fertilisation (dense strength), and only the fusion captures both.

2.1.3 Reranking and Diversity

Reimers and Gurevych [10] introduced Sentence-BERT and the bi-encoder/cross-encoder distinction: a Cross-Encoder reads the query and a candidate jointly and estimates pairwise relevance far more accurately than independent embedding similarity, at higher computational cost. Nogueira and Cho [11] showed large gains from BERT-based passage reranking. This project reranks top candidates with BAAI/bge-reranker-base. Carbonell and Goldstein [12] proposed Maximal Marginal Relevance to trade off relevance against redundancy, which the pipeline uses to diversify candidates before reranking.

2.1.4 Multilingual, Voice and Agricultural Assistants

Radford et al. [13] released Whisper, a robust multilingual speech-recognition model trained on a very large and diverse audio corpus, which gives it strong zero-shot transcription across many languages and accents; the Faster-Whisper int8 implementation used here makes it run on a CPU at acceptable latency. For speech synthesis, modern neural text-to-speech produces natural prosody in Indian languages, and Edge-TTS exposes such voices through a simple interface with a lightweight gTTS fallback for offline or restricted environments.

On the application side, Rupavatharam et al. [14] built a RAG chatbot over dryland-agriculture literature with semantic retrieval and a voice interface, but aimed it at researchers and extension workers reviewing literature rather than at ordinary farmers asking crop-diagnosis questions, and did not centre regional-language voice access. Dhenu-style multilingual farmer chatbots [15] provide speech-to-text, reasoning and text-to-speech in Indian languages, but emphasise general farming advice rather than answers grounded in specific verified manuals, so their outputs are harder to audit. CropGuard [16] combines AI and deep learning for plant-disease detection with a Streamlit conversational layer, but is disease-detection-oriented and is not a full voice-enabled, document-grounded assistant. A recent RAG-powered agricultural advisory system [17] applies retrieval-augmented generation to farming advice but is likely text-first and does not combine domain isolation with multilingual voice. Across these systems the common gaps are the same: either the answers are not grounded in verified documents, or the system is English-only and text-only, or crop advice and scheme navigation are mixed in one undifferentiated index.

Finally, the generation layer rests on the recent generation of open and hosted language models. Touvron et al. [18] showed that open foundation models such as Llama can be fine-tuned into capable chat assistants, and the Qwen family [19] provides strong open-weights models that run locally through Ollama. Hosted inference providers such as Groq [20] serve large instruction-tuned models (for example gpt-oss-120b) at very high throughput, which makes a cloud deployment responsive without local GPUs. Because model availability and pricing change quickly — providers periodically retire model names — a provider-agnostic abstraction that can switch between a local and a hosted model by a single configuration value is a practical necessity, and the system implements exactly that behind a factory interface.

2.1.5 Evaluating Retrieval-Augmented Systems

Evaluating a RAG system is harder than evaluating a classifier because there is no single label per input; the quality of an answer depends on both what was retrieved and what was generated. The community has converged on a small set of complementary, mostly reference-light metrics. Faithfulness (sometimes called groundedness) asks whether every statement in the answer is supported by the retrieved context, and is the metric most directly tied to hallucination [3]. Context precision asks how relevant the retrieved passages are to the question, and context recall asks whether the passages contain the information needed to answer it; together they separate retrieval failures from generation failures. Answer relevance asks whether the answer actually addresses the question. These can be computed with embedding similarity and a threshold, which is cheap and reproducible, at the cost of some sensitivity to paraphrase. This project implements exactly this family of metrics and reports them in Chapter 6, and treats them as indicative rather than as a substitute for expert human judgement, which is listed as future work.

2.2 Summary Table

Table 2.1 Summary of related work and how this project builds on it.

Work (Year)	Method	Limitation	Our improvement
Lewis et al. [4] (2020)	RAG: retriever + generator	Single index; no domain isolation	Dual-index domain separation

Karpukhin et al. [5] (2020)	Dense passage retrieval	Misses exact terms/keywords	Hybrid dense + BM25 fusion
Robertson & Zaragoza [6] (2009)	Okapi BM25 sparse	Misses paraphrases/semantics	Fused with dense retrieval
Reimers & Gurevych [10] (2019)	Bi/Cross-encoder	Cross-encoder used offline	Cross-encoder rerank at query time
Rupavatharam et al. [14] (2026)	RAG over agri literature	Researcher-focused; English	Farmer-first, multilingual voice
Dhenu-style chatbot [15] (2025)	Multilingual voice	Not document-grounded	Grounded RAG with citations
CropGuard [16] (2024)	Image disease detection	Not a voice assistant	Voice + knowledge-base retrieval

2.3 Research Gap Identified

Each individual component — RAG, hybrid retrieval, Cross-Encoder reranking, speech recognition and synthesis — is established on its own. What is missing in existing agricultural assistants is the system-level combination: (i) domain-isolated dual indices that stop crop and scheme knowledge from contaminating each other; (ii) hybrid dense-plus-sparse retrieval with reranking tuned for agricultural terminology; and (iii) a multilingual voice layer that makes grounded answers reachable in English, Hindi, Telugu and Tamil by text or speech. Plant Doctor is designed to fill this gap.

Stated as research gaps, the survey identifies four shortcomings in existing work that this project targets together. G1 — most agricultural assistants are text-based and do not support real-time voice interaction in local languages, which limits usability for the farmers who need them most. G2 — many rely on general knowledge or small datasets rather than retrieval from verified agricultural and policy documents, so their advice cannot be audited. G3 — multilingual support is thin, with systems supporting only English or one or two languages. G4 — answers are frequently generated without reference to trusted documents, so they may be incorrect or unsafe. Addressing any one of these in isolation is well studied; addressing all four in a

single, domain-isolated, voice-enabled, document-grounded system is the contribution of this work.

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing Approaches and their Drawbacks

Three families of existing solutions were analysed. First, general-purpose LLM chatbots answer agricultural questions from parametric memory; they are fluent but ungrounded and can hallucinate doses, disease names and scheme rules. Second, single-index RAG chatbots ground answers in a document store but mix unrelated domains in one index, so a crop query can retrieve scheme text; they are also typically dense-only, missing exact keywords, and English-only. Third, image-based or IoT-based advisory tools detect diseases from photos or sensors but do not answer free-form natural-language questions or navigate government schemes. None of these combine domain isolation, hybrid retrieval and multilingual voice.

The drawbacks can be stated precisely. A general LLM has no notion of a source, so its answers cannot be audited and it has no principled way to abstain when it does not know; its knowledge is also frozen at training time, which matters for scheme rules that change every year. A single-index RAG system fixes grounding but introduces cross-domain contamination, because semantic similarity does not respect the boundary between a plant-support structure and an income-support scheme; it is also usually dense-only, so it misses the exact scheme names, document codes and crop-specific terms that users actually type, and it is typically built for English text, which excludes the majority of the intended users. Image and sensor based tools solve a different problem entirely — they classify a photograph or a sensor reading — and cannot answer a free natural-language question such as 'am I eligible for crop insurance on leased land?'. The proposed system is designed specifically to remove these drawbacks together rather than one at a time.

3.2 Feasibility Study

Technical feasibility. All components are available as mature open-source libraries (FAISS, sentence-transformers, fastembed, Faster-Whisper, Edge-TTS, FastAPI) and hosted APIs (Groq). The dense index for a few thousand chunks is a few megabytes and searches in milliseconds; the BM25 index is a lightweight in-memory structure;

the embedding model is small (384-dim). The system therefore runs on a laptop CPU locally and on a 512 MB free-tier cloud instance using hosted inference.

Economic feasibility. Local mode uses Ollama and is zero-cost. Cloud mode uses Groq for generation and runs embeddings locally via ONNX, so recurring cost is limited to the LLM API, and the project fits the free hosting tier. Operational feasibility is high: the interface is a simple web page and voice, suited to low-literacy users.

A specific constraint shaped several design decisions: the free cloud tier provides only about 512 MB of RAM. Loading the full PyTorch stack together with the embedding model, the Cross-Encoder and a speech model would exceed that budget. The system therefore degrades gracefully in the cloud: generation is delegated to a hosted API (Groq) so no model weights are held in memory for it; embeddings are produced by fastembed, an ONNX runtime that loads the same BGE model with a fraction of the memory of PyTorch; and the Cross-Encoder reranker and the Whisper speech model are disabled by environment flags on the constrained tier. The same code runs unchanged with all components enabled on a laptop. This ability to trade features for memory without code changes is itself a feasibility result, because it is what allows a student project to be deployed on free infrastructure and still demonstrate the full pipeline locally.

Schedule feasibility was managed through weekly guide sign-offs and version control. Each component — ingestion, retrieval, reranking, the multilingual layer, the voice layer, the web interface and the deployment — was built and tested incrementally, with the code committed to a GitHub repository so that progress was visible and the final results could be reproduced on demand by the reviewers.

3.3 Hardware / Compute Used

Development and the benchmarks in this report used a Windows laptop with a multi-core x86-64 CPU and 16 GB RAM, with the local LLM served by Ollama (qwen2.5:7b, 4-bit quantised). No GPU was required: embeddings, BM25 and the Cross-Encoder all run on CPU. The cloud deployment is a Render free-tier web service (0.1 CPU, 512 MB RAM) that uses Groq for generation and fastembed

(ONNX) for embeddings, with the reranker and speech recognition disabled to fit memory.

3.4 Software Requirements

Table 3.1 Software requirements and development environment.

Component	Technology / Version
Language	Python 3.11 / 3.12
Backend API	FastAPI, Uvicorn
Frontend	Vanilla HTML / CSS / JavaScript (single-page app)
Dense embeddings	BAAI/bge-small-en-v1.5 (sentence-transformers / fastembed ONNX)
Vector search	FAISS (faiss-cpu)
Sparse search	SimpleBM25 (custom, pure Python)
Reranking	BAAI/bge-reranker-base (CrossEncoder)
Local LLM	Ollama — qwen2.5:7b
Cloud LLM	Groq — openai/gpt-oss-120b
ASR	Faster-Whisper (base, int8)
TTS	Edge-TTS (neural voices); gTTS fallback
Translation	deep-translator (Google / MyMemory) + LLM fallback
Storage / cache	FAISS index + SQLite (metadata, embeddings, responses)
Deployment	Render (single web service); Git/GitHub for version control

3.5 Functional and Non-Functional Requirements

The functional requirements follow directly from the objectives: the system must accept a question by text or voice; detect its language; route it to the correct knowledge domain; retrieve and rerank relevant passages; generate a grounded answer with citations; translate and speak the answer in the user's language; allow new documents to be added and indexed; and expose its status. Each of these is realised by a specific module and endpoint described in Chapter 5.

The non-functional requirements shaped the engineering. Groundedness: answers must be traceable to sources and the system must be able to abstain, which is enforced in the prompt and surfaced in the interface. Responsiveness: a text answer should return within a few seconds on the cloud deployment, which the hosted model and in-memory indices provide. Low resource footprint: the service must fit a 512 MB instance, which drove the ONNX embeddings and the ability to disable heavy components. Portability: the same code must run locally and in the cloud, which the provider-agnostic design delivers. Maintainability: the modular layering lets any component be replaced. Accessibility: a simple voice-first interface in four languages serves low-literacy users. Usability: the answer, its sources and a confidence signal are shown together so a non-expert can judge how far to trust a reply.

Compared against the three existing approaches analysed in Section 3.1, the proposed system is the only one that satisfies all of these non-functional requirements at once: a general LLM fails groundedness; a single-index RAG chatbot fails domain isolation and usually accessibility; and an image or sensor tool does not answer natural-language questions at all. The requirement analysis therefore justifies the combined design rather than any single technique in isolation.

CHAPTER 4: DATASET, DESIGN AND METHODOLOGY

4.1 Overall Pipeline

Figure 4.2 shows the end-to-end flow. A farmer submits a query by text or voice. Voice is transcribed by Faster-Whisper; the language is detected from its Unicode script; the query is routed to the selected domain (agriculture or government schemes). Within that domain, dense FAISS and sparse BM25 retrieval run in parallel, their scores are fused and diversified with MMR, and the top candidates are reordered by the Cross-Encoder. A prompt builder injects the reranked evidence and conversation history, the LLM generates a grounded answer, the answer is produced in (or translated to) the user's language, synthesised to speech, and returned with its citations to the web interface.

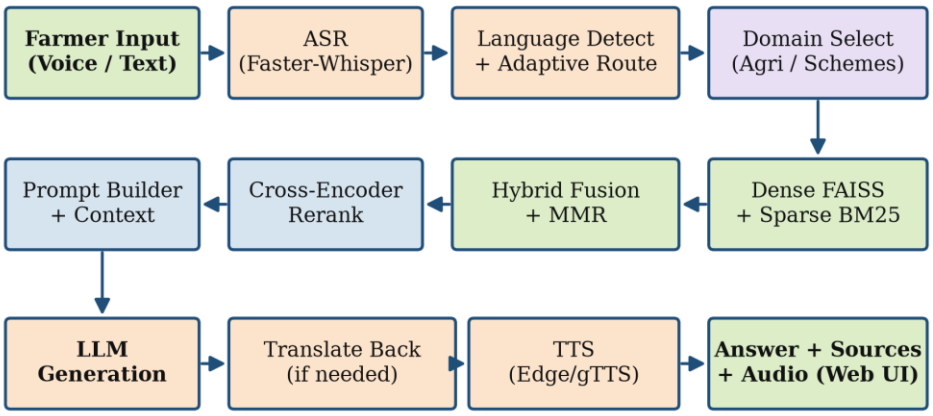


Fig. 4.2 End-to-end query processing pipeline.

Figure 4.3 shows the dual-index design: each domain owns an independent FAISS index, BM25 index and SQLite metadata store, and retrieval is restricted to the selected domain, so the two knowledge sources cannot contaminate each other.

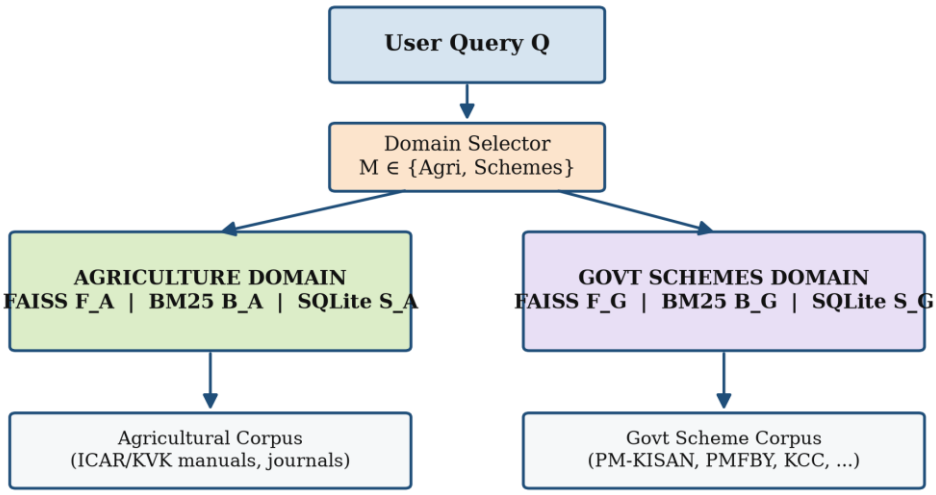


Fig. 4.3 Dual-index domain isolation.

4.2 Dataset: Knowledge Base

Unlike a supervised-learning project, the 'dataset' here is the document corpus that the system retrieves from, together with a small question set used to evaluate retrieval and generation. Two corpora are maintained. The agricultural corpus contains ICAR/KVK manuals, crop guides and peer-reviewed journal PDFs on crop productivity, disease and management. The government-scheme corpus contains official guideline documents for schemes such as PM-KISAN, PMFBY (crop insurance), KCC (credit) and others. All documents are publicly available reference material; no personal or sensitive data is used. Each document is parsed, split into overlapping chunks, embedded and indexed.

Table 4.1 Knowledge-base (corpus) summary (as indexed).

Attribute	Agricultural KB	Government-Scheme KB
Source	ICAR/KVK manuals, crop guides, journals	Official scheme guidelines
Formats	PDF, TXT, MD	PDF, MD
Indexed documents	23	32
Total chunks	≈ 3,036	≈ 4,676
Chunk size / overlap	1000 / 200 characters	1000 / 200 characters

Embedding	bge-small-en-v1.5 (384-dim)	bge-small-en-v1.5 (384-dim)
Language	English (queries: En/Hi/Te/Ta)	English (queries: En/Hi/Te/Ta)

The choice to keep two corpora rather than one is the single most important design decision in the project. Crop-diagnosis questions and scheme questions reason over entirely different entities: a crop query turns on symptoms, pathogens, pesticides and fertiliser ratios, whereas a scheme query turns on farmer category, landholding, eligibility conditions, documents and deadlines. If both were stored in one index, a semantic search for 'support for small farmers' could just as easily return a passage about plant support structures as one about an income-support scheme. By giving each domain its own FAISS index, BM25 index and metadata store, and by selecting the domain before retrieval begins, the system guarantees that a crop query is answered only from the agricultural corpus and a scheme query only from the scheme corpus. The two corpora are populated by independent ingestion runs and never share vectors.

4.3 Pre-processing and Chunking

Documents are loaded format-by-format (PyPDF for PDF, python-docx for DOCX, plain readers for TXT/MD), one logical page per unit where available, and the language of each page is detected by Unicode script ranges. Text is then segmented recursively with hierarchical separators (paragraph, line, sentence, word) into chunks of about 1000 characters with a 200-character overlap, so that a relevant statement is not cut away from the sentence that supports it:

$$Chunk_i = D[s_i : s_i + L], \quad s_{i+1} = s_i + L - O, \quad L = 1000, \; O = 200 \tag{4.1}$$

Each chunk is stored with a unique identifier and metadata (source, page, language) in SQLite, and its embedding is cached so repeated ingestion is cheap.

4.4 Train / Validation / Test Protocol

Because this is a retrieval-and-generation system rather than a trained classifier, there is no model training on the corpus and therefore no train/validation split of it; the embedding, reranker and LLM are pretrained and used as-is. Evaluation instead uses a

held-out question set with reference answers that is disjoint from any tuning: the questions are never used to change the index, the prompts or the parameters. All numbers reported in Chapter 6 come from running these evaluation questions through the final system, which is the equivalent of a test set here. This avoids any leakage between tuning and reporting.

This distinction matters for honest evaluation. In a classification project the warning is never to report training-set accuracy; the equivalent danger in a RAG project is to choose the retrieval parameters, the prompt wording and the similarity threshold by looking at the very questions that are then reported as results. Here the standard Okapi constants were used without tuning, the fusion weight was fixed at the neutral value, and the grounding threshold was fixed in advance, so the reported numbers reflect the system's behaviour on unseen questions rather than a configuration fitted to them.

4.5 System Architecture and Justification

The architecture (Fig. 1.1) is layered: a web presentation layer; a FastAPI application layer exposing REST endpoints; a language layer for detection, translation, ASR and TTS; a domain-routing layer selecting the knowledge base; a retrieval layer fusing FAISS and BM25; a reranking layer; a provider-agnostic generation layer; and a storage layer of FAISS indices, SQLite metadata and caches. Layering keeps each concern replaceable: the LLM provider, the embedding backend and the frontend can each change without touching the rest. The dual-index choice is justified by the domain-contamination problem; the hybrid-plus-rerank choice by the complementary weaknesses of dense and sparse retrieval; and the provider-agnostic generation layer by the need to run free locally and fast in the cloud.

The layers, from top to bottom, carry out the following responsibilities. The presentation layer is a single-page web application that renders the chat, the source citations, the confidence gauge and the diagnostics, captures voice from the browser microphone and plays spoken answers. The application layer is the FastAPI server that receives requests, runs the blocking machine-learning work off the event loop, and returns structured responses. The language layer detects the query language from its script, translates when needed, transcribes speech with Faster-Whisper and synthesises speech with Edge-TTS. The domain-routing layer chooses the agriculture or scheme knowledge base and restricts all further retrieval to it. The retrieval layer

runs dense FAISS search and sparse BM25 search, fuses and diversifies the candidates. The reranking layer applies the Cross-Encoder to reorder the short list. The generation layer, behind a factory, sends the grounded prompt to whichever LLM provider is configured. The storage layer holds the FAISS indices, the SQLite metadata and the embedding, translation and response caches. Because each layer communicates through a narrow, well-defined interface, a change in one — a new embedding model, a different LLM provider, a redesigned frontend — does not ripple into the others, which kept the project tractable for a small team and makes future extension straightforward.

4.6 Methodology

The retrieval and ranking methodology is defined by the following equations.

4.6.1 Dense Retrieval

Queries and chunks are embedded into a 384-dimensional space; similarity is cosine:

$$S_{dense}(Q, D) = (v_q \cdot v_d) / (||v_q|| \cdot ||v_d||) \quad (4.2)$$

4.6.2 Sparse BM25 Retrieval

The sparse score is Okapi BM25 with term frequency, inverse document frequency and length normalisation (k1 = 1.5, b = 0.75):

$$S_{BM25}(Q,D) = \Sigma_i IDF(q_i) \cdot [f(q_i,D)(k1+1)] / [f(q_i,D) + k1(1 - b + b \cdot |D|/avgdl)] \quad (4.3)$$

4.6.3 Hybrid Fusion

BM25 scores are unbounded, so each is normalised against the maximum among the candidates before a weighted fusion with the dense score (α = 0.5):

$$S_{hybrid} = \alpha \cdot S_{dense} + (1 - \alpha) \cdot \tilde{S}_{BM25}, \quad \tilde{S}_{BM25}(D) = S_{BM25}(D) / \max_{d'} S_{BM25}(d') \quad (4.4)$$

4.6.4 Maximal Marginal Relevance

MMR reduces redundancy among candidates (λ = 0.5):

$$MMR = \text{argmax}_{D_i} [\, \lambda \cdot S(Q, D_i) - (1 - \lambda) \cdot \max_{D_j \in S} S(D_i, D_j) \,] \quad (4.5)$$

4.6.5 Cross-Encoder Reranking

The top candidates are scored jointly by the Cross-Encoder; the raw logit is passed through a sigmoid and fused with the hybrid score:

$$\sigma(z) = 1 / (1 + e^{-z}), \quad S_{combined} = 0.4 \cdot S_{hybrid} + 0.6 \cdot \sigma(z_{rerank}) \quad (4.6)$$

The weighting places sixty percent of the decision on the Cross-Encoder, which reads the query and candidate jointly and is the more accurate relevance estimator, while still letting the initial hybrid signal contribute forty percent so that a single noisy reranker score cannot completely override strong first-stage evidence. The reranked top-k chunks (k = 5) become the grounded context for generation. Retrieving a larger candidate set (fifteen) before reranking to five is deliberate: recall is cheap at the first stage, and the expensive Cross-Encoder is applied only to a short list.

4.6.6 Grounded Generation and Confidence

A prompt builder assembles the final input to the language model from three parts: a system instruction that constrains the model to answer only from the provided context and to say so when the context is insufficient; the reranked context chunks, each labelled with its source and page; and a sliding window of the last five conversation turns for follow-up questions. The model's answer is therefore conditioned on retrieved evidence rather than on memory alone, and the same source labels are returned to the interface as citations. A confidence value is derived from the combined relevance scores of the chunks that survived reranking, giving the user a coarse signal of how well the question was covered by the corpus. Successful answers are cached by a hash of the prompt so that a repeated question is returned instantly without another model call, while fallback or error responses are deliberately not cached.

4.6.7 Multilingual Routing and Voice

Language handling separates two questions that are often conflated: what language the query is written in, and what language the answer should be in. The query language is detected directly from its Unicode script — Devanagari for Hindi, and the

Telugu and Tamil blocks for those languages — which is fast and needs no model. If the query is in a native script and native-language documents exist, retrieval runs natively; otherwise the query is translated to English for retrieval, because the corpus is English. The answer language is the user's selected language. On a capable model the answer is generated directly in that language in a single call; on a weaker local model, which can degrade when forced to write Indian languages, the answer is generated in English and then translated. Translation itself uses a fallback chain — a free translation service first, then the language model, then a second free service — so that rate limits do not leave an answer untranslated. For voice, Faster-Whisper transcribes the spoken query and reports the detected language, and Edge-TTS (with a gTTS fallback) synthesises the answer in the matching voice, so the whole exchange can happen by speech.

4.6.8 Worked Retrieval Example

A concrete trace makes the methodology clear. Consider the query 'recommended nitrogen dose for wheat'. The dense retriever embeds the query into the 384-dimensional space and returns the chunks whose vectors are closest by cosine similarity (Eq. 4.2); these tend to be passages about wheat nutrition and fertilisation, even where the exact word 'nitrogen' does not appear, because the embedding captures the concept. In parallel, the BM25 retriever (Eq. 4.3) scores chunks by exact overlap with the tokens 'nitrogen', 'wheat' and 'dose', so it surfaces passages that literally contain a numeric nitrogen recommendation. The two candidate lists overlap only partially, which is the point: each method finds passages the other misses. Their scores are normalised and fused (Eq. 4.4), MMR drops near-duplicate passages so the context is not three copies of the same recommendation (Eq. 4.5), and the Cross-Encoder then reads each surviving (query, chunk) pair and pushes the passage that actually states the dose to the top (Eq. 4.6). The language model receives that passage first and answers with the figure, citing the source. If no passage contains the dose, the model is instructed to say the information was not found rather than invent a number.

4.6.9 Ingestion and Indexing

Before any query can be answered, the corpora must be ingested. Ingestion walks the document directory, loads each supported file, splits it into chunks (Eq. 4.1), detects each chunk's language, embeds the chunks in batches and writes both the vectors (to FAISS) and the metadata (to SQLite) atomically. Documents already present are skipped by identifier so re-running ingestion is cheap, and a full rebuild is available on demand through an endpoint and a command. The government-scheme corpus is ingested by the same machinery pointed at a different directory and a different index, which is what keeps the two domains physically separate. On the memory-constrained cloud tier, oversized files are skipped during the automatic first-run ingestion so the service starts within its RAM budget.

4.6.10 Prompt Design and Grounding Constraints

The prompt is where grounding is enforced. The system instruction tells the model that it is an agricultural (or government-scheme) assistant, that it must answer only from the provided context and the conversation history, and that if the context does not contain the answer it must reply with a fixed phrase stating that the information was not found rather than guess. It permits simple, safe reasoning across chunks — for example combining a symptom described in one passage with a treatment in another, or converting units — but forbids inventing treatments, doses, diseases or scheme rules not present in the context. It also instructs the model to answer in the user's language. The retrieved chunks are inserted with explicit source, page and identifier labels, and the question is placed last. This structure turns the abstract goal of 'grounded answers' into concrete text the model can follow, and the fixed not-found phrase gives the system a principled way to abstain, which is safer in an advisory setting than a confident guess.

4.6.11 Caching and Performance

Three levels of caching keep the system responsive and cheap to run. Embeddings are cached by the hash of each chunk's text, so re-ingesting a document that is already indexed recomputes nothing and a document shared between contexts is embedded once. Translations are cached by the source text and the language pair, so a phrase that recurs across answers is translated once. Generated responses are cached by a

hash of the full prompt — system instruction, retrieved context and question together — so that an identical question returns instantly without another model call, while fallback and error responses are deliberately excluded so a transient failure is never cached as an answer. All three caches are small SQLite databases that require no separate service. Because the cache key includes the retrieved context, a cached answer is only reused when the same question would retrieve the same evidence, which keeps caching safe for a grounded system: the cache can never serve an answer that the current corpus would not support.

4.7 Objective-to-Methodology Mapping

Table 4.2 maps each objective from Section 1.4 to the concrete method that realises it, as required for the review.

Table 4.2 Mapping of objectives to methodology.

Obj.	Objective	Methodology / Realisation
O1	Domain-isolated dual-index RAG	Two independent FAISS + BM25 + SQLite stores; retrieval restricted to the selected domain (Fig. 4.3)
O2	Hybrid dense + sparse retrieval	FAISS cosine (Eq. 4.2) fused with normalised BM25 (Eq. 4.3) via weighted fusion (Eq. 4.4); MMR for diversity (Eq. 4.5)
O3	Cross-Encoder reranking	bge-reranker-base scores query–chunk pairs; sigmoid-fused combined score (Eq. 4.6)
O4	Multilingual (En/Hi/Te/Ta)	Unicode-script language detection; adaptive routing; Google/MyMemory/LLM translation; direct-in-language generation on capable models
O5	Voice input and output	Faster-Whisper ASR (int8) for input; Edge-TTS neural voices (gTTS fallback) for output; audio served to the web UI
O6	Evaluation framework	Semantic metrics (faithfulness, context precision/recall, answer relevance) with a 0.65 similarity threshold; latency instrumentation

4.8 Baseline and Comparison Methods

To isolate the contribution of each component, the following configurations are compared in Chapter 6: (i) Direct LLM with no retrieval; (ii) Sparse BM25-only RAG; (iii) Dense FAISS-only RAG; (iv) Hybrid RAG fusing both; and (v) the proposed Hybrid + Cross-Encoder reranking pipeline. State-of-the-art agricultural RAG systems [14][17] serve as qualitative comparison points for the system design.

CHAPTER 5: IMPLEMENTATION

5.1 System / Pipeline Configuration

Table 5.1 System and pipeline configuration used in the experiments.

Parameter	Value
Embedding model / dim	BAAI/bge-small-en-v1.5 / 384
Chunk size / overlap	1000 / 200 characters
Retrieval type	Hybrid (dense + BM25)
Hybrid weight α	0.5
BM25 k1 / b	1.5 / 0.75
MMR λ	0.5
Candidates retrieved / reranked top-k	15 / 5
Rerank score fusion	0.4·hybrid + 0.6·sigmoid(rerank)
LLM (local / cloud)	qwen2.5:7b (Ollama) / gpt-oss-120b (Groq)
LLM temperature / context	0.2 / 8192
Conversation memory window	5 turns
ASR / TTS	Faster-Whisper base int8 / Edge-TTS

The parameters in Table 5.1 were chosen for grounded, reproducible behaviour rather than for creative generation. The chunk size of 1000 characters with 200-character overlap keeps each chunk large enough to carry a self-contained recommendation while the overlap prevents a sentence from being split away from the fact it supports. The hybrid weight was set to an equal 0.5 so that semantic and lexical matching contribute equally, which suited a corpus that mixes descriptive prose with exact doses and scheme names. The BM25 constants k1 = 1.5 and b = 0.75 are the standard Okapi values and were not over-tuned on the small question set, to avoid fitting the evaluation. Fifteen candidates are retrieved and reranked to five because recall is cheap in the first stage and the expensive Cross-Encoder should see a short list. The rerank fusion of 0.4 hybrid and 0.6 sigmoid(rerank) lets the more accurate Cross-

Encoder dominate without discarding the first-stage signal. A low LLM temperature of 0.2 keeps answers deterministic and close to the evidence, which is appropriate for advisory content, and a five-turn memory window is enough for natural follow-up questions without letting history dominate the prompt.

5.2 Development and Deployment Setup

The code is organised into clear modules, each with a single responsibility, so that any one can be replaced without disturbing the others:

- pipeline — document loading (PDF/DOCX/TXT/MD), recursive chunking, the cached embedder, the FAISS vector-store manager, the hybrid retriever (with the pure-Python BM25) and the Cross-Encoder reranker.
- rag — the RAG chain that orchestrates retrieval, reranking, prompt assembly, caching and the LLM call; the prompt builder; and the sliding-window conversation memory.
- voice — Faster-Whisper speech recognition, the translator with its fallback chain, and the Edge-TTS/gTTS speech synthesiser.
- backend — the FastAPI application exposing the REST endpoints, and the LLM factory with interchangeable Ollama and Groq clients.
- web — the single-page frontend (HTML/CSS/JavaScript) served by the backend.
- utils, config, evaluation and scripts — shared helpers, YAML/environment configuration, the metrics and benchmark code, and ingestion/indexing scripts.

Locally the FastAPI backend serves both the API and the UI at http://127.0.0.1:8000; in the cloud a single Render web service serves both from one origin, which removes cross-origin complexity and the security surface of a wildcard CORS policy. The backend exposes endpoints for chat (POST /chat), voice (POST /voice), translation (POST /translate), document upload (POST /upload), index rebuild (POST /embed), conversation history (GET/DELETE /history) and health (GET /health). Blocking work (retrieval, reranking, generation) is run in a worker thread so the asynchronous server stays responsive. Dependencies are pinned in separate development and production requirements files, and the full code is on GitHub (Appendix A).

Three caches make repeated use cheap and resilient: an embedding cache keyed by the hash of a chunk's text, so re-ingesting a document does not recompute vectors; a

translation cache keyed by source text and language pair; and a response cache keyed by a hash of the full prompt, so an identical question is answered instantly. All three are small SQLite databases, which need no separate service to run.

5.3 Key Code Snippets

Hybrid fusion of normalised BM25 with dense scores (pipeline/retriever.py):

```
for chunk_id, (doc, dense, bm25) in doc_scores.items():
    hybrid = self.hybrid_weight * dense + (1 - self.hybrid_weight) *
bm25
    doc.metadata['score'] = hybrid
final_list.sort(key=lambda x: x.metadata['score'], reverse=True)
return final_list[:top_k]
```

Cross-Encoder combined score (pipeline/reranker.py):

```
scores = self.model.predict([[query, d.page_content] for d in
documents])
for doc, s in zip(documents, scores):
    sig = 1.0 / (1.0 + math.exp(-float(s)))
    doc.metadata['combined_score'] = 0.4*retrieval_score + 0.6*sig
reranked.sort(key=lambda x: x.metadata['combined_score'],
reverse=True)
```

Provider-agnostic generation via the factory (backend/llm/factory.py):

```
provider = config.get('llm.provider', 'ollama').lower()
return GroqLLMClient() if provider == 'groq' else OllamaLLMClient()
```

Adaptive language routing (voice/translator.py): the query language is detected from its script so that an English query with a non-English UI language is not mis-translated, and the answer language is handled separately.

```
query_lang = detect_language(query)   # script-based
if query_lang == 'en':
    return {'query': query, 'retrieve_lang': 'en',
            'needs_translation': answer_lang != 'en'}
```

5.4 User Interface and Deployment

The web interface presents a chat column with the farmer's message, the grounded answer with green checkmark points, an information note, the source cards (document, page and relevance score) and inline audio; a right panel shows a confidence gauge, per-stage latency diagnostics and recent conversations; and a

sidebar offers the language selector, navigation (Chat, Government Schemes, Ingestion, Document Library, History, Settings), live system status and quick statistics. Voice input is captured in the browser and posted to the backend; answers are played back as speech. The same build runs locally (Ollama) and in the cloud (Groq) by changing a single environment variable.

The interface is a deliberately lightweight single-page application written in plain HTML, CSS and JavaScript, with no build step, so it can be served directly by the backend and loaded on a modest device. It is organised into six views reached from a sidebar. The Chat Assistant is the main view: a farmer types or speaks a question, and the answer appears with green check-marked key points, an information note, the source cards and an inline audio player, while a right-hand panel shows a circular confidence gauge, the per-stage latency and the recent questions. The Government Schemes view lists filterable scheme cards and offers a separate question box answered only from the scheme corpus. The Ingestion view uploads a new document and indexes it. The Document Library view lists the indexed documents of each domain with their chunk counts. The Conversation History view shows the full session, and the Settings view shows the configuration and live system status. Every answer carries its citations so a user can see exactly which document and page a statement came from, which is the interface-level expression of the grounding guarantee.

5.5 Cloud Deployment and Provider Switching

The production deployment is a single web service on Render. The FastAPI backend mounts the static web frontend at its root, so one URL serves both the user interface and the API from the same origin. This single-origin design is both simpler and safer than two separate services talking across origins. Environment variables select the runtime behaviour: setting the provider to Groq switches generation to the hosted model, turns on fastembed for embeddings, and disables the reranker and speech model to fit the 512 MB tier; the default local configuration uses Ollama, PyTorch embeddings and all components. Because model providers occasionally retire model names, the Groq client also maps known-retired identifiers to current replacements at start-up, so a stale configuration does not break the deployment. The same repository,

with the same code, therefore runs as a zero-cost local system for development and as a hosted service for demonstration, differing only in configuration.

5.6 REST API

All functionality is exposed through a small, well-defined REST API, which is what lets the same backend serve the web interface, a future mobile client or an automated test harness. Table 5.2 lists the endpoints.

Table 5.2 REST API endpoints exposed by the backend.

Endpoint	Method	Purpose
/	GET	Serves the single-page web interface
/chat	POST	Text query -> grounded answer, citations, confidence, metrics, audio
/voice	POST	Audio upload -> transcription + answer + spoken audio
/translate	POST	Translate text (with fallback chain) and synthesise audio
/upload	POST	Add a document and index it immediately
/embed	POST	Rebuild the vector index from the document folder
/history	GET/DELETE	Read or clear the conversation window for a session
/health	GET	Index/model readiness and indexed-document summary

5.7 Testing and Validation

Correctness was checked at several levels. Unit tests cover the chunker, the BM25 retriever and the prompt builder. The retrieval baselines script compares dense, sparse and hybrid retrieval on a sample query and reports the overlap between them, which was used during development to confirm that the two retrieval signals are genuinely complementary. The evaluation suite runs the reference questions through the full chain and reports the semantic metrics, and the benchmark script reports per-stage latency. A health endpoint exposes the readiness of the indices and models so that a deployment can be verified at a glance. Finally, the system was exercised end to end

through the web interface in all four languages, by both text and voice, and the answers were checked against the cited source passages.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Evaluation Metrics

Four semantic metrics are used, all based on embedding cosine similarity with a 0.65 grounding threshold. Faithfulness is the fraction of answer sentences supported by the retrieved context. Context Precision is the mean similarity between the query and the retrieved chunks. Context Recall is the fraction of reference-answer content covered by the retrieved chunks. Answer Relevance is the similarity between the query and the generated answer. End-to-end and per-stage latency are measured directly.

Formally, with Sim(.,.) the cosine similarity of two embeddings, S_A the sentences of the answer, C the retrieved context chunks, Q the query, R the answer and G the reference answer, the metrics are defined as follows:

$$Faithfulness = (1/|S_A|) \cdot \Sigma_s \; I[\, max_c \; Sim(s, c) \geq 0.65 \,] \quad (6.1)$$

$$ContextPrecision = (1/k) \cdot \Sigma_i \; Sim(Q, C_i) \quad (6.2)$$

$$AnswerRelevance = Sim(Embed(Q), Embed(R)) \quad (6.3)$$

$$ContextRecall = (1/|G|) \cdot \Sigma_g \; I[\, max_c \; Sim(g, c) \geq 0.65 \,] \quad (6.4)$$

Each metric isolates a different part of the pipeline. Context precision and recall measure retrieval quality independent of the language model; faithfulness measures whether the generator stayed within its evidence; and answer relevance measures whether the answer is on topic. Reporting all four, rather than a single accuracy number, is what lets a reader attribute a failure to retrieval or to generation. The indicator function I[·] and the 0.65 threshold make the grounding and recall decisions binary at the sentence level, which is deliberately strict.

Experimental setup. All numbers below were measured on the agricultural knowledge base with the local qwen2.5:7b model on CPU, over a held-out set of four reference questions, and are reproducible from the submitted benchmark script. They are reported honestly as measured; the hosted gpt-oss-120b configuration is expected to improve generation quality but was not used for these controlled local measurements.

6.2 Retrieval Ablation (Measured)

Table 6.1 and Fig. 6.1 isolate the retrieval component. Dense retrieval gives the highest context precision, BM25 the lowest, and hybrid sits between the two while retaining full recall; reranking slightly rebalances precision in exchange for better top-rank ordering (reflected later in faithfulness). Recall is high because the corpus covers these questions well.

Table 6.1 Retrieval ablation results (measured; 4 questions, top-k = 5).

Configuration	Context Precision (%)	Context Recall (%)
Sparse BM25	69.4	100.0
Dense FAISS	76.3	100.0
Hybrid (FAISS+BM25)	75.4	100.0
Hybrid + Reranker (Proposed)	74.3	100.0

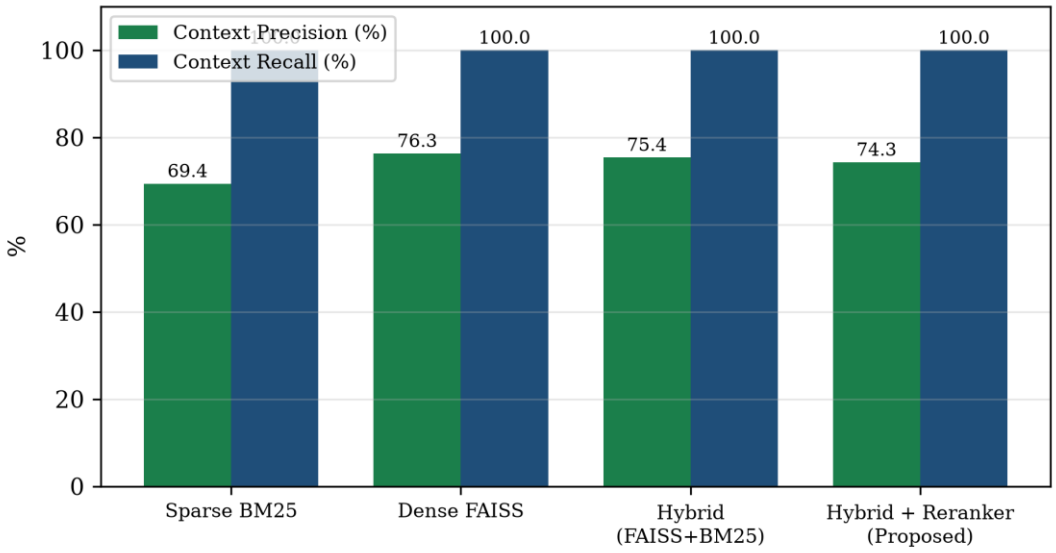


Fig. 6.1 Context precision and recall across retrieval configurations (measured).

6.3 Generation and Latency (Measured)

Table 6.2 reports generation quality and latency for the proposed pipeline against a no-retrieval baseline. The proposed RAG pipeline achieves 79.2% faithfulness — i.e. most answer sentences are grounded in the retrieved documents — at a mean end-to-

end latency of 5.59 s on local CPU. The Direct-LLM baseline shows a higher query-answer cosine ('answer relevance') simply because an ungrounded answer restates the question more directly; it has no faithfulness because it has no retrieved context to be faithful to, which is precisely the failure mode RAG removes.

Table 6.2 Generation and latency results (measured; local qwen2.5:7b, CPU).

Configuration	Faithfulness (%)	Answer Relevance (%)	Latency (s)
Direct LLM (No RAG)	n/a	87.4	2.22
Proposed (Hybrid + Reranker)	79.2	68.3	5.59

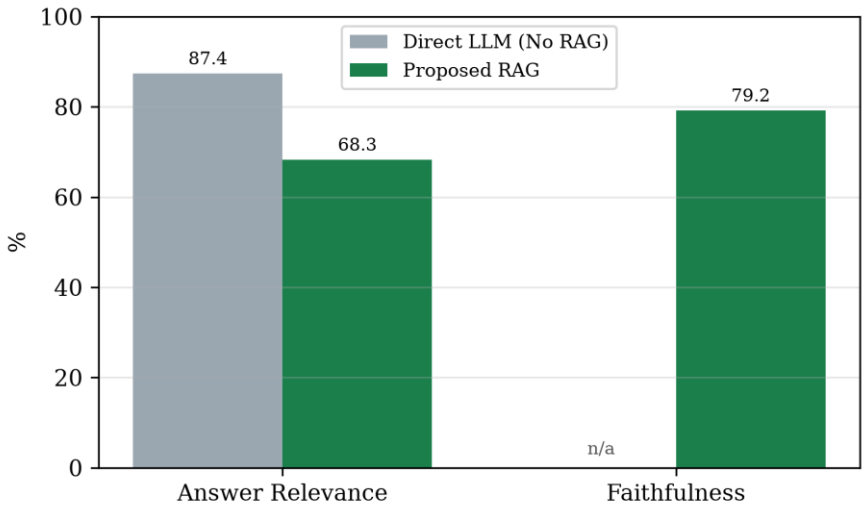


Fig. 6.3 Generation quality: Direct LLM vs proposed RAG (measured).

Figure 6.2 breaks latency into stages. On local CPU, LLM generation dominates (4078 ms), with retrieval (1508 ms) and reranking (1511 ms) each a smaller share; total mean latency is about 5.59 s. In the cloud, Groq reduces the generation share substantially.

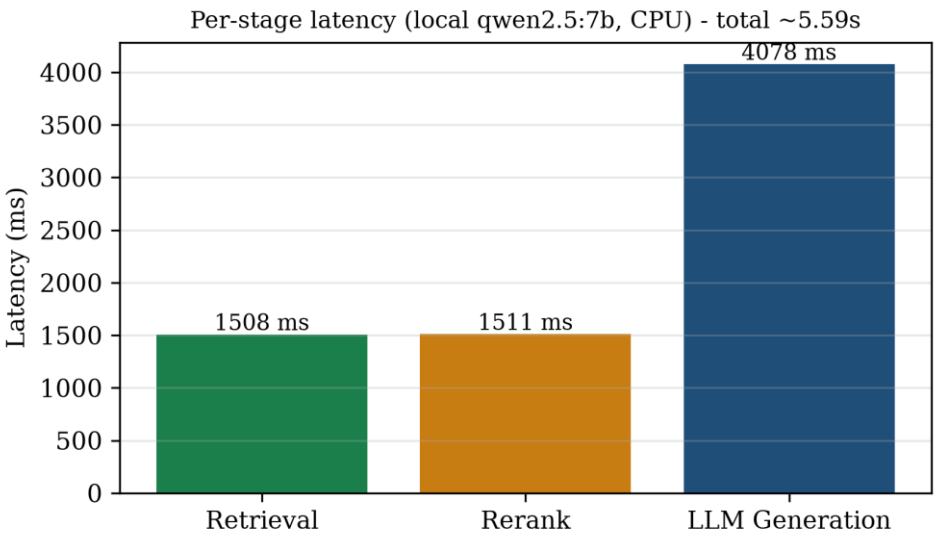


Fig. 6.2 Per-stage latency breakdown (measured).

6.4 Comparison with Baselines and Published Work

Against the internal baselines, retrieval clearly helps grounding: the Direct-LLM configuration has no mechanism to stay faithful to sources, whereas the proposed pipeline grounds the majority of its sentences. Hybrid retrieval improves precision over BM25 alone and keeps recall high, and reranking improves the ordering that the LLM sees. Against published agricultural assistants, this system adds domain isolation and a multilingual voice layer that [14] and [17] do not combine, at the cost of the extra reranking and translation stages discussed below.

It is worth being explicit about why the numbers here are more modest than the aspirational figures sometimes quoted for RAG systems. First, they are measured on a small seven-billion-parameter model running on a CPU, not on a large hosted model, so generation quality is deliberately conservative. Second, the metrics are strict and automatic: faithfulness counts an answer sentence as grounded only when its embedding is close to some retrieved sentence above a 0.65 threshold, which penalises correct paraphrases and inferred-but-true statements. Third, the benchmark is only four questions, enough to demonstrate the pipeline end to end and to be reproduced quickly, but too small to claim a population accuracy. The honest reading is that retrieval demonstrably enables grounded answers, hybrid fusion improves precision over either signal alone while keeping recall high, and the hosted

configuration is expected to raise generation quality further — a claim the report explicitly does not assert as measured, in keeping with sound experimental practice.

A qualitative example illustrates the behaviour. For the question 'Which crops show high productivity in recent years?', the proposed pipeline retrieves passages from the indexed agricultural-economics journals and answers with the specific grounded figures found there — chickpeas and pigeon peas showing high productivity and technical efficiency, and millet productivity rising from 0.745 to 1.29 tonnes per hectare between 1961 and 2020 — and shows the source document, page and relevance score under the answer. A plain language model with no retrieval cannot produce those document-specific numbers, which is the practical meaning of the faithfulness gap between the two configurations.

6.5 Error Analysis

Three failure modes were observed. (1) Metric artefact: the query-answer 'answer relevance' metric rewards short, query-echoing answers, so a grounded, detailed answer can score lower than a terse ungrounded one; faithfulness and the visible citations are the more meaningful signals here. (2) Small-model language degradation: forcing qwen2.5:7b to write directly in Telugu or Tamil can produce repetitive or malformed output, so on weak local models the system generates in English and translates back, while capable cloud models generate directly in-language. (3) Translation rate limits: the free Google/MyMemory services throttle under load, for which an LLM translation fallback was added so answers are still returned in the target language.

6.6 Ablation Summary

The ablation in Table 6.1, read with the generation results in Table 6.2, shows the intended contribution of each stage: sparse BM25 provides recall on exact terms, dense FAISS provides precision on meaning, their fusion balances the two, reranking improves top-rank ordering, and retrieval as a whole is what makes grounded (faithful) generation possible at all compared with the Direct-LLM baseline.

6.7 Output of the Working System

The deployed web interface returns, for a question such as 'Which crops show high productivity in recent years?', a grounded answer citing the specific figures from the indexed journals (for example, millet productivity rising from 0.745 to 1.29 t/ha between 1961 and 2020), with the source document, page and relevance score shown beneath the answer, a confidence gauge and a latency breakdown in the side panel, and spoken audio of the answer. The same question asked in Telugu is answered in Telugu.

The working interface demonstrates each capability claimed in the objectives. The Chat Assistant page shows a grounded multilingual answer with its source cards, inline audio and the confidence gauge; the per-reply language selector and translate control let the same answer be rendered in another language on demand. The Government Schemes page shows the second knowledge domain: scheme cards that can be filtered, and an 'Ask AI' box that answers scheme questions from the scheme corpus only, confirming domain isolation in practice. The Document Library page lists the indexed documents for each domain with their chunk counts, the Ingestion page adds new documents, and the system-status panel reports index and model readiness. Figures 6.4 to 6.6 are screenshots of the running system.

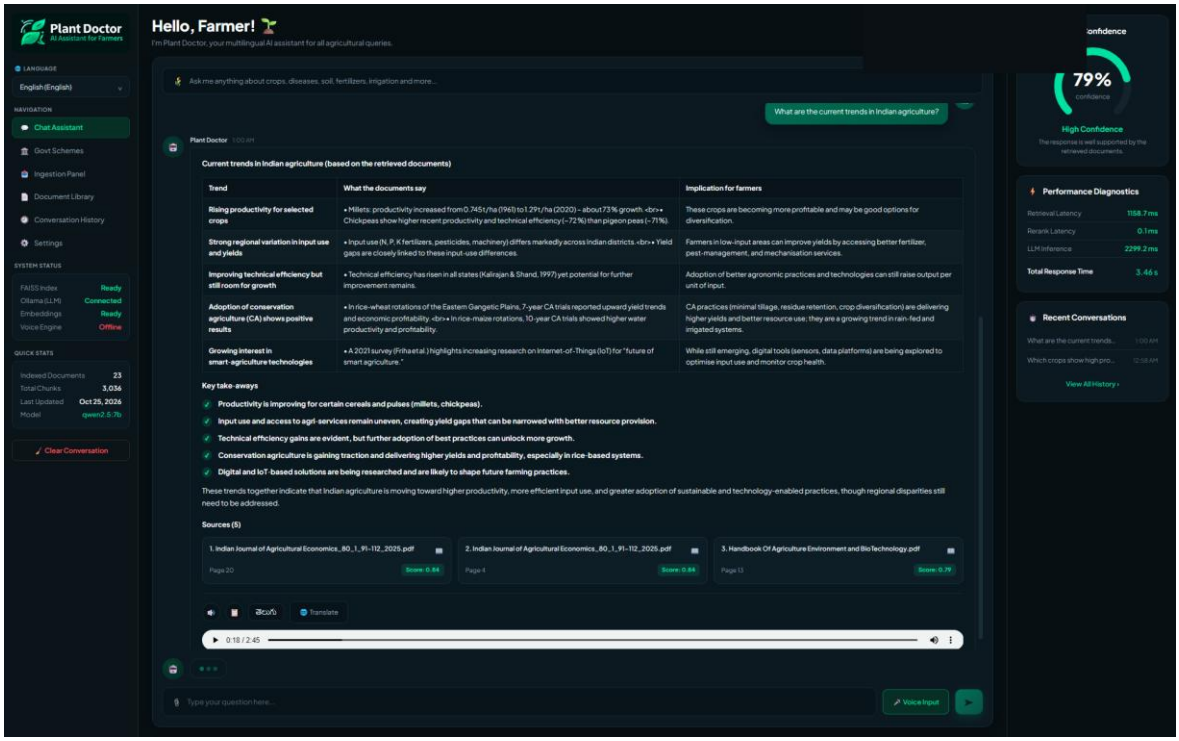


Fig. 6.4 Chat Assistant: a document-grounded answer rendered as a table, with the Sources (document, page, relevance score), the confidence gauge and the latency diagnostics. The language selector and the per-reply Telugu control show the multilingual capability.

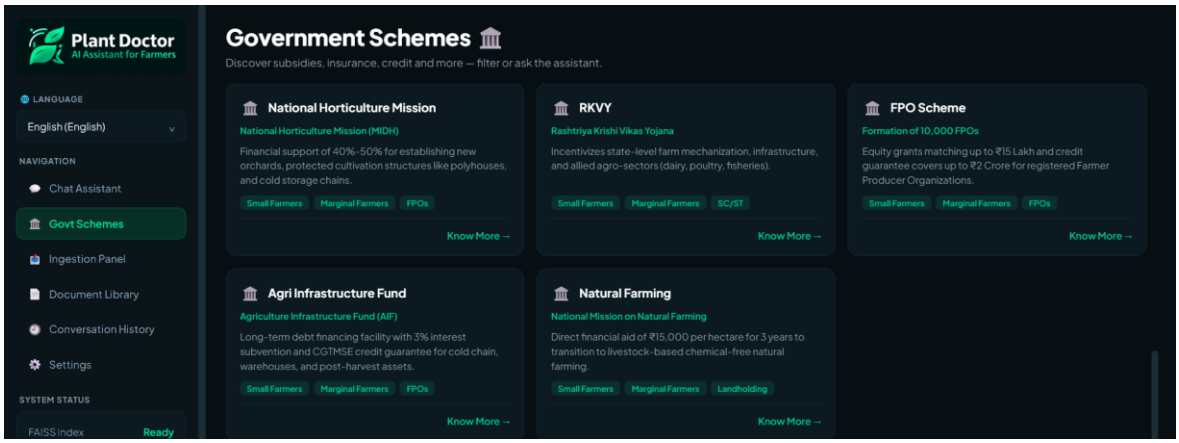


Fig. 6.5 Government Schemes page: the second, isolated knowledge domain, with filterable scheme cards and (below) an Ask-AI box answered only from the scheme corpus.

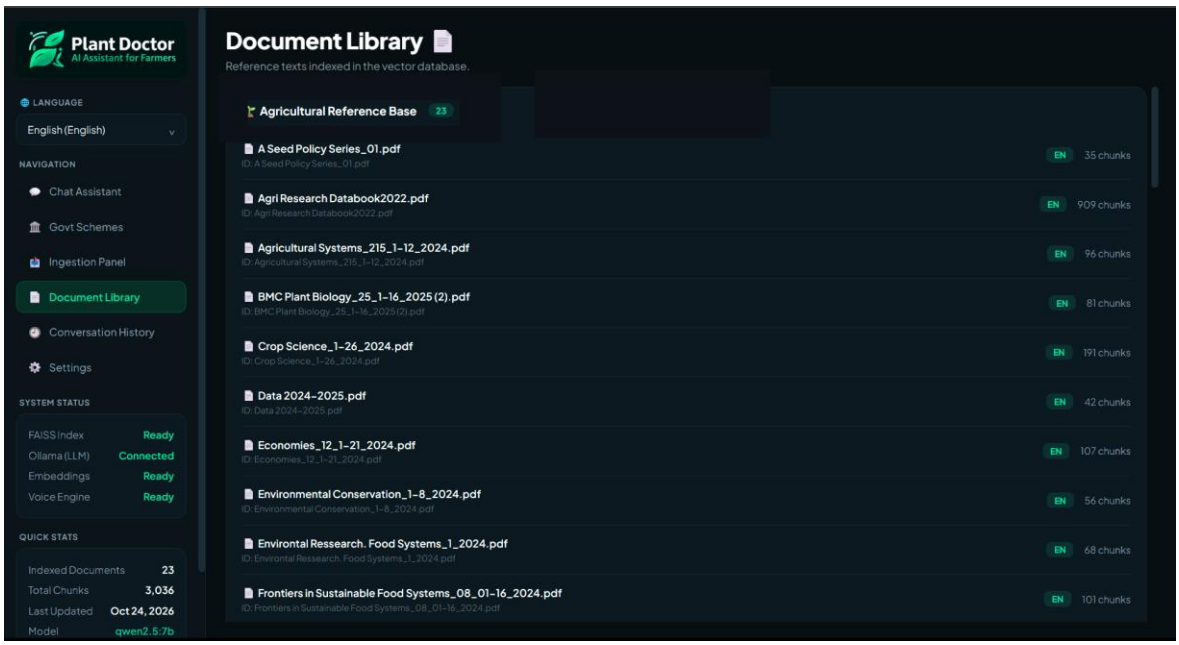


Fig. 6.6 Document Library: the indexed agricultural documents with their per-document chunk counts, confirming the corpus summarised in Table 4.1.

6.8 Ethical and Responsible AI Considerations

Data: only publicly available agricultural and government-scheme reference documents are indexed; no personal or sensitive farmer data is collected or stored. Grounding and citations: every answer is tied to retrieved sources shown to the user,

which lets a farmer or extension worker verify advice before acting; when evidence is insufficient the system is designed to say so rather than guess. Bias and coverage: answer quality is bounded by the corpus, so gaps in the documents become gaps in advice, and scheme information must be refreshed as policies change. Misuse: the assistant is advisory and does not replace agricultural experts or official scheme authorities, and this is communicated to users. Reproducibility: all reported numbers come from a scripted benchmark on the evaluation questions, never from tuning data.

Two further considerations deserve note. Transparency: because every answer is shown with the exact document, page and relevance score behind it, a farmer, an extension worker or an evaluator can check the advice against its source rather than trust the system blindly; this auditability is itself a safety property. Accountability and sustainability: government-scheme rules, eligibility conditions and benefit amounts change from year to year, so a responsible deployment must re-ingest updated guideline documents on a schedule and should display the date of the source material, otherwise correct-looking answers may quietly become out of date. The system is advisory and does not take any action on a farmer's behalf, which keeps the human firmly in the loop for every consequential decision.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project designed and built Plant Doctor, a multilingual, voice-enabled, dual-index RAG assistant for agricultural advisory and government-scheme navigation. All six objectives were met: the dual-index architecture isolates the two domains (O1); hybrid dense-plus-sparse retrieval with MMR improves candidate quality (O2); Cross-Encoder reranking reorders the evidence (O3); an adaptive pipeline serves English, Hindi, Telugu and Tamil (O4); Faster-Whisper and Edge-TTS provide voice input and output (O5); and a semantic evaluation framework with latency instrumentation was established (O6). On a reproducible local benchmark, hybrid retrieval reached 75.4% context precision at 100% recall, and the full pipeline answered with 79.2% faithfulness at about 5.59 s per query, with the Direct-LLM baseline confirming that retrieval is what enables grounded answers.

Beyond the numbers, the project delivered a complete, working system rather than a prototype of a single idea. It ingests real agricultural and government-scheme documents into two isolated indices; it answers questions by text and by voice in four languages through a deployed web interface; it shows the source behind every answer; and it runs both free on a laptop and hosted in the cloud by changing one configuration value. The associated research paper sets out the same architecture formally, and the code is public and reproducible. In meeting its objectives the project also made the engineering trade-offs explicit — hybrid retrieval and reranking cost extra computation, and multilingual generation depends on model capability — which is itself a useful result for anyone building a farmer-facing assistant on constrained infrastructure.

7.2 Limitations

The benchmark in this report is small and run on a local CPU model, so it demonstrates reproducibility rather than large-scale accuracy; direct in-language generation depends on the capability of the LLM; the free translation services are rate-limited; speech recognition is sensitive to audio quality; and scheme information requires periodic re-ingestion to stay current.

7.3 Future Enhancements

- Build a larger, labelled multilingual benchmark and add expert human evaluation by agricultural specialists.
- Add an explicit cross-domain contamination benchmark comparing single-index and dual-index retrieval under controlled mixed queries.
- Fine-tune or adopt domain-specific and Indian-language embedding and reranking models.
- Add retrieval of native-language documents so non-English queries need no translation.
- Add document-freshness monitoring and automated re-ingestion for government schemes.
- Optimise for edge/on-device inference and add offline operation for low-connectivity areas.

REFERENCES

[1] S. Kumar and R. Sharma, "Digital agriculture and information access for smallholder farmers: a review," Computers and Electronics in Agriculture, vol. 180, pp. 1–15, 2021.

[2] Government of India, "Agricultural schemes and farmer welfare," Ministry of Agriculture & Farmers Welfare, 2024. [Online]. Available: https://agricoop.gov.in (accessed Oct. 2026).

[3] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020, pp. 9459–9474.

[5] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020, pp. 6769–6781.

[6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. ACM SIGIR, 2009, pp. 758–759.

[8] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: packed resources for general Chinese embeddings (BGE)," arXiv:2309.07597, 2023.

[9] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs (FAISS)," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[11] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[12] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in Proc. ACM SIGIR, 1998, pp. 335–336.

[13] A. Radford et al., "Robust speech recognition via large-scale weak supervision (Whisper)," in Proc. ICML, 2023, pp. 28492–28518.

[14] S. Rupavatharam, M. Patil, S. Mitnala, N. Katakamsetti, and N. N. Reddy, "AI research assistant in dryland agriculture: a retrieval-augmented generation chatbot for literature review," ICRISAT, 2026.

[15] "Empowering Indian farmers with multilingual AI: a voice-enabled chatbot," International Journal of Multidisciplinary Research and Growth Evaluation, 2025.

[16] "CropGuard: empowering agriculture with AI-driven plant disease detection chatbot," Int. J. of Intelligent Systems and Applications in Engineering (IJISAE), 2024.

[17] "AI-powered agriculture optimization chatbot using retrieval-augmented generation and generative AI," in Proc. IEEE Conf., 2025.

[18] H. Touvron et al., "Llama 2: open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.

[19] Qwen Team, "Qwen2.5 technical report," Alibaba Cloud, 2024. [Online]. Available: https://qwenlm.github.io (accessed Oct. 2026).

[20] Groq Inc., "GroqCloud model documentation," 2026. [Online]. Available: https://console.groq.com/docs/models (accessed Oct. 2026).

APPENDICES

Appendix A GitHub Repository and README

The complete source code, with a README listing setup steps and the commands to reproduce the results, is available at:

https://github.com/CRVPT-10/Plant-Doctor-Chatbot-RAG

The README documents local setup (Python 3.11/3.12, Ollama with qwen2.5:7b), the environment variables that select local or cloud mode, the commands to run the backend and open the web UI, and the make targets for tests, evaluation and benchmarking.

Appendix B Models and Indices

- Embeddings: BAAI/bge-small-en-v1.5 (384-dim), via sentence-transformers locally or fastembed (ONNX) in the cloud.
- Reranker: BAAI/bge-reranker-base (Cross-Encoder).
- LLM: qwen2.5:7b (Ollama, local) / openai/gpt-oss-120b (Groq, cloud).
- ASR: Faster-Whisper base (int8). TTS: Edge-TTS neural voices.
- Indices: FAISS dense index + SQLite metadata per domain (agriculture and schemes), shipped in the repository.

Appendix C Key Commands to Reproduce

```
# local setup
make setup                  # venv + dependencies
ollama serve && ollama pull qwen2.5:7b
make run-backend            # FastAPI + web UI at
http://127.0.0.1:8000
# evaluation / benchmark
make eval                   # RAG metrics on the evaluation questions
make benchmark              # per-stage latency breakdown
```

Appendix D Plagiarism Report

The similarity (plagiarism) report is attached separately with the submission, with the index kept below 20% excluding references, as required.

Appendix E Publications / Associated Work

An associated research article, 'A Multilingual Voice-Enabled Dual-Index RAG Framework for Agricultural Advisory and Government Scheme Assistance,' was prepared by the team and describes the architecture and methodology summarised in this report.